

# Choosing Between Arrays and Linked Lists

Programming and Data Structures — Week 4, Lecture 4

Dr. Innocent Nyalala

## Objectives

By the end of this lecture, you will be able to choose between an array and a linked list for a given, concretely described workload and justify that choice using this course's own complexity results rather than folklore; explain why the common claim "linked lists are slower" is an oversimplification that misses real cases where a linked list is the better choice; implement a sparse polynomial using a linked list and explain precisely why an array-based representation would waste memory in that setting; and trace through a simplified Least Recently Used (LRU) cache, connecting its core mechanism directly to Lecture 2's  $O(1)$  doubly linked deletion.

## Outline

Today covers a librarian analogy for choosing storage strategy by access pattern; the complete decision table consolidating every complexity result from this week and Week 2; why the common claim that linked lists are categorically slower misses real, common cases; a worked application, sparse polynomial representation; a second worked application, a simplified LRU cache; the MTech-track preview of skip lists; an in-class quiz; and a summary.

## The librarian analogy

Picture two librarians with different jobs. One shelves books by call number and needs to jump directly to any shelf on demand — this librarian wants array-like, direct-address access. The other processes a pile of just-returned books, only ever handling the one on top — this librarian wants stack-like access, exactly Week 3's LIFO discipline. Neither approach is categorically better; the correct choice depends entirely on the actual pattern of access a given task requires, which is precisely the lesson this closing lecture consolidates across the whole course so far.

## The complete decision table

This table consolidates every complexity result established across Week 2 and this week. No single structure wins every row: arrays dominate index access; linked lists (of either kind) dominate front insertion and deletion; doubly linked lists specifically dominate back operations and reference-based deletion; and search by value is  $O(n)$  for all three, since none of them maintain any ordering that would allow a faster search without additional structure (Week 7's trees address exactly this gap). The bottom row is worth taking seriously: a linked list's flexibility is not free, it is paid for in extra memory per element.

## Why “linked lists are slower” misses real cases

The blanket claim “linked lists are slower” is true for some access patterns and false for others, which is precisely why it is worth retiring as an unqualified rule. It is true for random index access and for appending when no `tail` reference is maintained — arrays clearly win both. It is false for a workload dominated by front-insertion and front-deletion, where a linked list's  $O(1)$  beats an array's  $O(n)$  outright. It is also false whenever an application frequently needs to delete “the item I am currently holding a reference to” — a doubly linked list wins this case outright, structurally, not marginally.

## A worked cost estimate: 10,000 front-insertions

Putting real numbers on the decision table's front-insertion row: performing 10,000 consecutive front-insertions into an array requires, in total, roughly  $1 + 2 + \dots + 10,000 \approx 50,000,000$  element moves, since each insertion shifts every existing element. The equivalent operation on a linked list requires exactly 2 reference updates per insertion, for a total of 20,000 updates across all 10,000 operations — a difference of roughly 2,500 times less total work for this specific, front-insertion-heavy workload. This is the kind of concrete number worth having in mind before dismissing the choice of structure as a minor implementation detail.

## Application: sparse polynomials

Consider representing a polynomial like  $x^{1000} + 3x^2 + 1$ , which has only 3 non-zero terms despite its highest exponent being 1000. An array representation indexed by exponent would require 1001 slots, with 998 of them wastefully storing a zero coefficient. A linked list, storing one node per non-zero term only, uses memory proportional to the actual number of terms present, completely independent of how large the highest exponent happens to be — exactly the kind of workload where a linked list is the clearly superior choice, not merely an acceptable alternative.

## Sparse polynomial, in code

```
class Term:
    def __init__(self, coeff, exponent):
        self.coeff = coeff
        self.exponent = exponent
        self.next = None

class Polynomial:
    def __init__(self):
        self.head = None      # terms kept in decreasing order of exponent

    def add_term(self, coeff, exponent):
        new_term = Term(coeff, exponent)
        if self.head is None or exponent > self.head.exponent:
            new_term.next = self.head
            self.head = new_term
            return
        current = self.head
        while current.next and current.next.exponent > exponent:
            current = current.next
        new_term.next = current.next
        current.next = new_term
```

Each Term node holds a coefficient and an exponent, and the polynomial keeps terms sorted in decreasing order of exponent — an insertion-sort-style placement, inserting each new term at its correct sorted position exactly as Week 2 Lecture 4’s sorted-array insertion did conceptually, but here without the  $O(n)$  shifting cost, since linking a node in requires only reference updates, not moving existing elements.

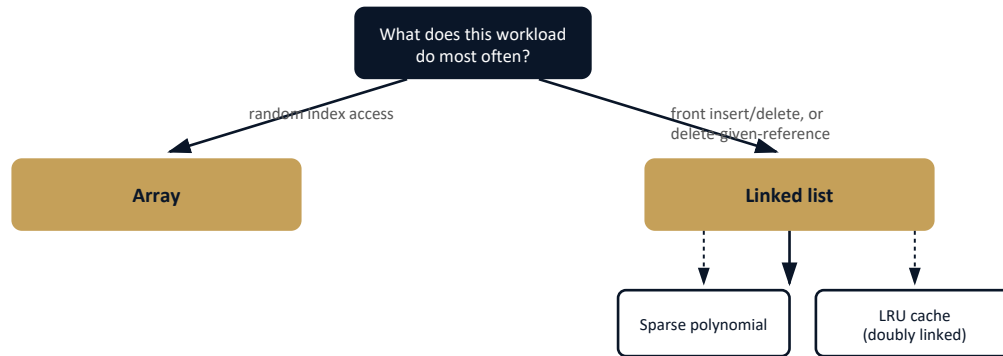
### Tracing polynomial construction for $x^{1000} + 3x^2 + 1$

```
add_term(1, 1000):  head is None → head = [1x^1000]
add_term(3, 2):      2 < 1000 → traverse: current=[1x^1000], current.next is None → insert after
                    Ring: [1x^1000] → [3x^2]
add_term(1, 0):      0 < 2 → traverse: current=[3x^2], current.next is None → insert after
                    Chain: [1x^1000] → [3x^2] → [1x^0]
```

Tracing the three `add_term` calls needed to build  $x^{1000} + 3x^2 + 1$ : each term finds its correct sorted position by exponent, using only the three non-zero terms actually present — the linked structure

never allocates anything corresponding to the 998 absent terms between exponent 2 and exponent 1000. Evaluating or printing this polynomial simply traverses the three-node chain, exactly as Lecture 1's traversal pattern established.

### The picture: two workloads, two winning structures



*The correct structure follows from the dominant access pattern of the actual workload, not from either structure's general reputation.*

This diagram is this lecture's central idea in visual form: the correct structure follows directly from the dominant access pattern of the actual workload, not from a general reputation either structure has acquired.

### Application: a simplified LRU cache

A Least Recently Used cache holds a fixed number of items, and when full, evicts whichever item has gone the longest without being accessed to make room for a new one. This requires two operations, both needing to be fast: moving an item to the "most recently used" end whenever it is accessed, and removing the item at the "least recently used" end when eviction is needed. A doubly linked list provides both in  $O(1)$ , using precisely Lecture 2's  $O(1)$  deletion-given-a-node-reference — the cache keeps a dictionary mapping keys directly to their node, so "move to front" is delete-then-reinsert, both  $O(1)$  given the node reference already in hand.

### The LRU cache, sketched in code

```

class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.list = DoublyLinkedList()      # Lecture 2's structure
        self.node_map = {}                  # key -> node, for O(1) lookup

    def access(self, key):
        if key in self.node_map:
            node = self.node_map[key]
            self.list.remove_node(node)      # Lecture 2: O(1), given the reference
        new_node = self.list.push_back_node(key)  # re-insert at the "recent" end
        self.node_map[key] = new_node
        if len(self.list) > self.capacity:
            evicted = self.list.head         # Lecture 2: head is "least recent"
            self.list.remove_node(evicted)
            del self.node_map[evicted.value]

```

This sketch combines Week 8's hash table idea (a dictionary for  $O(1)$  key lookup, covered formally in two weeks) with this week's doubly linked list. `access` moves a key to the "recent" end by removing its existing node (if present) and reinserting at the tail, both  $O(1)$  using Lecture 2's machinery; eviction, when the cache exceeds capacity, removes the head — the least recently used item — also  $O(1)$ . This is a preview of a genuinely important real-world data structure; a full implementation returns in Week 8 once hash tables are covered formally.

### Tracing the LRU cache on a real sequence

Tracing a capacity-2 LRU cache through four accesses: A and B are added normally. Accessing A again, which is already present, removes its existing node and reinserts it at the "most recent" end — this is the "move to front" operation, using Lecture 2's  $O(1)$  delete-given-a-reference directly. Accessing C pushes the cache over capacity, triggering eviction of whatever sits at the "least recent" end — B, which was accessed longest ago — leaving the cache holding A and C.

### MTech addition — skip lists, a preview

A plain linked list's fundamental weakness, visible throughout this week, is that it cannot support anything like binary search's  $O(\log n)$ , even if kept sorted, because reaching the middle element still requires  $O(n)$  traversal — there is no direct-address jump the way an array's index formula provides. A skip list addresses this by adding extra "express lane" references at higher levels, each skipping over several nodes at once, allowing a search to jump ahead in large strides before dropping to finer-grained links near the target. This buys expected  $O(\log n)$  search while retaining  $O(1)$  insertion, at the cost of

the extra express-lane pointers' memory — a structure occasionally used in real systems (Redis's sorted sets, for instance) and worth knowing exists, though a full implementation is beyond this course's scope.

#### **Week 4 in one table: what each lecture added**

Reviewing the week as a single progression: Lecture 1 established the singly linked list's core trade-off,  $O(1)$  front operations at the cost of  $O(n)$  index access. Lecture 2 added a `prev` reference and an explicit `tail`, buying  $O(1)$  back operations and, critically,  $O(1)$  deletion given a node reference. Lecture 3 introduced circularity, a structural change with its own traversal hazards, solving the Josephus problem and connecting to round-robin scheduling. This closing lecture ties all three together into a single decision framework, backed by two genuine applications — sparse polynomials and LRU caches — where a linked structure is demonstrably the right choice, not merely an acceptable one.

#### **Exercise**

As an exercise, implement `Polynomial.evaluate(x)`, computing the value of the polynomial at a given  $x$  by traversing the term chain and summing `coeff * x**exponent` for each term. Separately, implement `Polynomial.add(other)`, merging two polynomials' sorted term chains into a single result, combining coefficients where exponents match — this is structurally the same “merge two sorted sequences” idea formalized as merge sort's core step in Week 6.

#### **A note on premature optimization**

A closing caution: this week's decision table matters when performance genuinely matters at the scale a workload actually operates at. For small  $n$  — tens or low hundreds of elements — the difference between an  $O(1)$  and an  $O(n)$  operation is frequently a matter of microseconds, invisible to a user and often not worth the added implementation complexity of choosing the “theoretically optimal” structure. The habit established in Week 2 of measuring actual behavior before drawing conclusions applies here too: profile a real program at its real scale first, and let that evidence, not asymptotic notation alone, decide whether a structural change is actually worth making.

#### **Quiz — choose the structure**

For each of these three workloads, choose the most appropriate structure — array, singly linked list, or doubly linked list — and justify the choice using this lecture's decision table, before checking the answer: a music player's “next track” and “previous track” navigation; a fixed list of 100 students' final grades, read constantly by roll number and never resized; and an undo history that only ever undoes the single most recent action, with no redo capability.

## Quiz — answer

A music player's next/previous navigation is a direct match for a doubly linked list, needing exactly the bidirectional traversal Lecture 2 introduced. A fixed, read-heavy list of 100 grades accessed by roll number is best served by a plain array, since  $O(1)$  index access is exactly what matters and the list is never resized, so none of the linked structures' advantages apply. An undo history limited to only the single most recent action, with no redo, is naturally a stack — Week 3's LIFO structure, typically implemented with a singly linked list or a dynamic array, since no backward traversal beyond “undo the last thing” is ever needed.

## Summary

No single structure covered this week or in Week 2 wins on every operation — the decision table consolidated in this lecture replaces vague folklore with the actual, previously-proven trade-offs. The claim “linked lists are slower” is true for some access patterns and false for others, and knowing which applies to a given workload is the actual skill this course is building, not memorizing a blanket rule. Sparse polynomials and LRU caches are genuine applications where a linked structure is the better choice, not merely an acceptable alternative to an array. Skip lists preview a further trade-off worth knowing exists: extra memory spent on express-lane pointers in exchange for expected  $O(\log n)$  search on a linked structure. This closes Week 4 and the block of foundational data structures — OOP, arrays, stacks and queues, and linked lists. Next week turns to sorting algorithms, the first algorithms this course studies purely for their own properties, rather than as the internal mechanism of a data structure.